Dossier de lancement (semaine 1)

Nov 9, 2026 · @Larhlimi

Mode d'emploi

La semaine 1 est décisive : sur un projet de 4 semaines, un cadrage flou ou des données indisponibles ne se rattrapent pas. Ce dossier permet à l'enseignant de valider, faire reprendre ou refuser le projet sous 48 heures.

Le dossier comporte quatre livrables. **L'équipe se les répartit : chaque membre rédige seul le ou les livrables dont il est responsable, puis un autre membre le relit.** Le nom du rédacteur et le relecteur figure en tête de chaque livrable ; c'est une preuve de contribution pour la note individuelle.

Répartition selon la taille de l'équipe

Taille	Membre A	Membre B	Membre C	Membre D
2 étudiants	1. Descriptif + 3. Maquette	2. Dataset + 4. Pipeline		
3 étudiants	1. Descriptif + 4. Pipeline (consolidation)	2. Dataset	3. Maquette	
4 étudiants	1. Descriptif	2. Dataset	3. Maquette	4. Pipeline (consolidation)

Le pipeline des tâches (livrable 4) est consolidé par un seul membre, mais **chaque membre y ajoute lui-même ses propres tâches**.

Notre répartition

Livrable	Rédacteur	Relecteur	Fichier dans le dépôt
1. Descriptif du projet	Kajaj Fatima Ezzahra	Hajar Elkhaldi	`docs/semaine1/1-descriptif.md`
2. Dataset	Hajar Elkhaldi	Ikram Sakhi	`docs/semaine1/2-dataset.md` + `data/sample/`
3. Maquette globale	Ikram Sakhi	Kajaj Fatima Ezzahra	`docs/semaine1/3-maquette.md` + `docs/semaine1/img/`
4. Pipeline des tâches	Kajaj Fatima Ezzahra (consolidation)	Hajar Elkhaldi	`docs/semaine1/4-pipeline.md`

Circuit GitHub

1. Le coordinateur crée le dossier `docs/semaine1/` avec un `README.md` qui reprend le tableau « Notre répartition ».
2. Chaque rédacteur copie la section de son livrable dans le fichier indiqué, travaille sur sa propre branche (`semaine1/dataset` , par exemple) et ouvre une pull request.
3. Le relecteur vérifie les critères d'acceptation du livrable, commente dans la pull request, puis l'approuve.
4. Une pull request n'est fusionnée qu'après approbation. Les quatre livrables sont fusionnés sur la branche principale avant la date limite.
5. Le coordinateur pose l'étiquette `semaine1-remis` sur le dernier commit et en informe l'enseignant.

Livrable 1 : descriptif du projet mis à jour

Rédacteur : Kajaj Fatima Ezzahra Relecteur : Hajar Elkhaldi Fichier : `docs/semaine1/1-descriptif.md`

Le descriptif adapte la fiche du catalogue à votre version du projet. Il doit pouvoir être lu seul par quelqu'un qui ne connaît pas le catalogue.

1.1 Contexte et problème (5 lignes au plus) : Les équipes de développement perdent du temps à diagnostiquer les échecs de pipelines CI/CD : lire des logs volumineux, corréler des erreurs entre étapes, chercher la cause racine dans la documentation ou les tickets. Aujourd'hui, un ingénieur parcourt manuellement les logs et les fichiers de configuration, ce qui prend souvent 30 à 60 minutes par incident.

1.2 Objectif en une phrase : Fournir un agent qui analyse les logs d'un pipeline CI/CD échoué et propose en moins de 30 secondes un diagnostic (cause probable) accompagné des extraits de logs et de la documentation pertinents, avec des suggestions de correction.

1.3 Utilisateurs cibles et scénario d'usage principal : Utilisateur : développeur ou ingénieur DevOps. Scénario :

1. Le développeur colle l'URL ou le contenu d'un log de pipeline échoué dans l'interface.
2. L'agent ingère le log, identifie les erreurs (exceptions, codes de sortie, tests échoués).
3. Il interroge une base de connaissances (documentation, runbooks, incidents passés) via recherche vectorielle.
4. Il renvoie un diagnostic structuré : cause probable, passages de logs cités, documents sources, étapes de correction suggérées.

1.4 Fonctionnalités du MVP (3 pour un trinôme)

N°	Fonctionnalité	Ce que l'utilisateur voit ou obtient	Priorité (indispensable, souhaitable)
F1	Ingestion et analyse de logs	Colle un log, l'agent en extrait les erreurs et les segmente par étape du pipeline	Indispensable
F2	Diagnostic avec citations	Un diagnostic de cause probable, avec extraits de logs et documents sources cités	Indispensable
F3	Interface web de soumission et d'historique	Formulaire de soumission + liste des diagnostics précédents	Souhaitable

1.5 Services et composants techniques

Service ou composant	Rôle dans le projet	Technologie choisie	Pourquoi ce choix
Modèle (LLM ou ML)	Générer le diagnostic à partir des logs et passages	API LLM (ex. OpenAI/Mistral) ou modèle local via Ollama	Qualité de raisonnement sur du texte technique ; local possible pour la confidentialité
Recherche ou mémoire (RAG, base vectorielle, SGBD)	Retrouver documentation/runbooks pertinents	ChromaDB + embeddings (sentence-transformers)	Léger, s'installe en local, adapté à un petit corpus
Outils appelés par l'agent	Parser les logs, extraire les erreurs	Python (`re` , parser custom)	Contrôle fin du prétraitement des logs
API	Exposer l'agent	FastAPI	Standard, rapide, documentation auto
Interface	Soumettre un log et lire le diagnostic	Streamlit	Prototypage rapide d'une UI
Conteneur et intégration continue	Déploiement et tests	Docker, GitHub Actions	Reproductibilité et CI sur chaque PR

1.6 Hors périmètre (ce que vous ne ferez pas) : Pas d'intégration directe aux plateformes CI (GitHub Actions, GitLab CI) via API ; pas de correction automatique du code ; pas de gestion multi-utilisateurs ni de droits ; pas d'analyse de logs en temps réel (uniquement soumission manuelle).

1.7 Écart avec la fiche du catalogue : La fiche catalogue proposait un agent de recherche documentaire générique. Nous le spécialisons sur les logs DevOps : ajout du parsing structuré des logs et d'un corpus de runbooks/incidents ; retrait de l'ingestion de PDF scannés.

Critères d'acceptation (vérifiés par le relecteur)

- [x] Le problème et l'utilisateur sont compréhensibles sans le catalogue.
- [x] Le nombre de fonctionnalités correspond à la taille de l'équipe.
- [x] Chaque fonctionnalité est démontrable (pas de formulation vague comme « améliorer »).
- [x] Chaque composant technique a une justification d'une ligne.
- [x] Le hors périmètre est rempli.

Livrable 2 : dataset

Rédacteur : Hajar Elkhaldi Relecteur : Ikram Sakhi Fichiers : `docs/semaine1/2-dataset.md + data/sample/`

2.1 Description

Rubrique	Réponse
Nom et source (lien)	Logs publics de projets open source (ex. builds GitHub Actions échoués de dépôts publics) + documentation/runbooks publics (ex. docs GitHub Actions, GitLab CI)
Producteur ou propriétaire	Communauté open source / éditeurs des plateformes CI
Volume (nombre de fichiers, lignes, pages, taille)	~100 fichiers de logs (≈ 10 000 lignes) + ~50 pages de documentation
Format et structure (PDF, CSV, JSON, logs, images ; colonnes principales)	Logs `.txt` / `.log` (lignes horodatées, étapes, erreurs) ; docs Markdown
Langue et période couverte	Anglais (majoritaire) et français, 2023–2026
Licence et droits d'usage	Logs publics sous licence des dépôts (MIT/Apache) ; docs sous licence ouverte (CC BY)
Données personnelles ou sensibles, et traitement prévu	Aucune ; anonymisation des noms d'utilisateurs/URLs internes si nécessaire
Emplacement des données complètes	Dépôt Git (échantillon < 10 Mo) ; corpus complet sur un stockage partagé indiqué dans le README

2.2 Qualité constatée sur l'échantillon : Logs hétérogènes (formats différents selon la plateforme), présence de lignes de bruit (timestamps, couleurs ANSI), doublons possibles entre builds, quelques logs tronqués. Documentation globalement propre.

2.3 Justification

Fonctionnalité	Données utilisées	En quoi elles suffisent
F1	Logs d'échecs réels	Contiennent des erreurs typiques à extraire et segmenter
F2	Logs + documentation/runbooks	Permettent de citer des passages et de fonder un diagnostic
F3	Historique des soumissions	Généré par l'application elle-même (pas de données externes)

2.4 Alternative de repli : Si les logs publics deviennent inaccessibles : générer des logs simulés à partir de scénarios d'échec typiques (tests échoués, dépendance manquante, timeout) + documentation officielle publique. Perte : moins de variété et de réalisme.

2.5 Jeu de test : 20 à 30 cas : chaque cas = un log échoué + cause attendue + documents/passages attendus, rédigés par l'équipe (Hajar Elkhaldi et Ikram Sakhi), validés en revue.

Critères d'acceptation (vérifiés par le relecteur)

- [x] L'échantillon est dans `data/sample/` et s'ouvre sans erreur.
- [x] La licence autorise l'usage dans un projet de cours.
- [x] Les données personnelles éventuelles sont anonymisées ou remplacées par des données fictives.
- [x] Chaque fonctionnalité du MVP est couverte par la justification.
- [x] L'alternative de repli est réellement accessible (lien vérifié).
- [x] Le jeu de test est défini, même partiellement rédigé.

Livrable 3 : maquette globale

Rédacteur : Ikram Sakhi Relecteur : Kajaj Fatima Ezzahra Fichiers : `docs/semaine1/3-maquette.md + docs/semaine1/img/`

3.1 Schéma d'architecture.

flowchart LR U[Utilisateur] --> UI[Interface Streamlit] UI -->|log soumis| API[API FastAPI /diagnose] API --> P[Parser de logs] P -->|erreurs extraites| R[Recherche vectorielle] R --> DB[(Base vectorielle Chroma)] ING[Ingestion docs/runbooks] --> DB R -->|passages| LLM[LLM] P -->|erreurs| LLM LLM -->|diagnostic + citations| API API --> HIST[(Historique)] API --> UI **3.2 Légende du schéma**

Composant	Fonctionnalité servie
Interface Streamlit	F1, F3
API FastAPI	F1, F2, F3
Parser de logs	F1
Recherche vectorielle + Chroma	F2
LLM	F2
Ingestion docs/runbooks	F2
Historique	F3

3.3 Maquette des écrans.

Écran	Image (`img/...`)	Fonctionnalités affichées	Action principale de l'utilisateur
Soumission de log	`img/ecran-soumission.png`	F1	Coller le log, lancer l'analyse
Diagnostic	`img/ecran-diagnostic.png`	F2	Lire cause + citations, copier la correction
Historique	`img/ecran-historique.png`	F3	Consulter un diagnostic passé

3.4 Parcours utilisateur : Écran Soumission → action « Analyser » → Écran Diagnostic (cause + citations) → retour possible vers Historique.

Critères d'acceptation (vérifiés par le relecteur)

- [x] Chaque composant du livrable 1 apparaît dans le schéma, et inversement.
- [x] Le parcours d'une requête se suit de bout en bout sur le schéma.
- [x] Chaque fonctionnalité du MVP est visible sur au moins un écran.
- [x] Le schéma s'affiche correctement sur GitHub (Mermaid valide ou image présente).
- [x] Pour une équipe de 4 : les agents ou sous-systèmes et leurs échanges sont représentés.

Livrable 4 : pipeline des tâches prévues

Consolidateur : Kajaj Fatima Ezzahra Relecteur : Hajar Elkhaldi Fichier : `docs/semaine1/4-pipeline.md`

Le pipeline découpe le projet en tâches de 1 à 2 jours. **Chaque membre saisit lui-même ses tâches** ; le consolidateur vérifie les dépendances et l'équilibre de la charge. Les tâches sont aussi créées comme issues GitHub (ou dans un tableau GitHub Projects) pour être suivies dans le rapport hebdomadaire.

4.1 Liste des tâches

ID	Tâche	Fonctionnalité	Responsable	Dépend de	Semaine	Terminée quand ? (critère vérifiable)
T1	Dépôt, environnement, squelette FastAPI/Streamlit	—	Kajaj Fatima Ezzahra	—	1	`docker compose up` lance les deux services
T2	Collecte et nettoyage des logs + docs	F1/F2	Hajar Elkhaldi	T1	1	50 logs + 20 docs dans `data/`
T3	Parser de logs (extraction d'erreurs)	F1	Hajar Elkhaldi	T2	2	10 logs de test parsés sans erreur
T4	Indexation vectorielle du corpus	F2	Ikram Sakhi	T2	2	Requête renvoie 5 passages en < 2 s
T5	Endpoint `/diagnose` + prompt LLM	F2	Kajaj Fatima Ezzahra	T3, T4	2	Réponse JSON avec citations sur 5 cas
T6	Interface Streamlit (soumission, diagnostic, historique)	F1/F3	Ikram Sakhi	T5	3	3 écrans fonctionnels
T7	Jeu de test (20 cas) et évaluation	F2	Hajar Elkhaldi	T5	3	20 cas rédigés + script de score
T8	Docker, CI GitHub Actions, README, démo	—	Kajaj Fatima Ezzahra	T6, T7	3	CI verte, image publiée, démo enregistrée

4.2 Jalons hebdomadaires (repris de la fiche de cadrage)

Semaine	Jalon démontrable	Tâches qui y mènent
1	Dossier de lancement fusionné, prototype en script	T1, T2
2	Agent exposé par l'API, premiers tests	T3, T4, T5
3	MVP complet, Docker et CI au vert	T6, T7, T8
4	Résultats mesurés, code gelé, soutenance	Évaluation, rédaction

4.3 Vue chronologique.

gantt dateFormat YYYY-MM-DD title Pipeline Agent DevOps section Semaine 1 T1 Dépôt et environnement :t1, 2026-11-16, 1d T2 Collecte et nettoyage données :t2, after t1, 2d section Semaine 2 T3 Parser de logs :t3, after t2, 2d T4 Indexation vectorielle :t4, after t2, 2d T5 Endpoint /diagnose :t5, after t3, 2d section Semaine 3 T6 Interface Streamlit :t6, after t5, 2d T7 Jeu de test et évaluation :t7, after t5, 2d T8 Docker, CI, démo :t8, after t6, 2d **4.4 Chemin critique et charge**

- Chaîne de tâches dont le moindre retard décale la soutenance : **T1 → T2 → T3 → T5 → T6 → T8**
- Charge par membre (nombre de jours de tâches) : Kajaj Fatima Ezzahra = 4 j (T1, T5, T8), Hajar Elkhaldi = 4 j (T2, T3, T7), Ikram Sakhi = 4 j (T4, T6) → écart ≤ 1 j.

Critères d'acceptation (vérifiés par le relecteur)

- [x] Aucune tâche ne dépasse 2 jours.
- [x] Chaque fonctionnalité du MVP a au moins une tâche, et chaque tâche a un critère de fin vérifiable.
- [x] Le MVP est complet au plus tard en fin de semaine 3.
- [x] Chaque membre a saisi ses propres tâches ; l'écart de charge entre membres reste inférieur à 2 jours.

- [x] Les tâches existent comme issues GitHub.

Remise et décision de l'enseignant

Checklist de remise (coordinateur)

- [x] `docs/semaine1/README.md` avec la répartition rédacteurs et relecteurs
- [x] Les quatre livrables fusionnés sur la branche principale, chacun par une pull request approuvée par son relecteur
- [x] Échantillon de données dans `data/sample/`
- [x] Schéma et écrans visibles sur GitHub
- [x] Issues GitHub créées pour les tâches du pipeline
- [x] Étiquette `semaine1-remis` posée et enseignant prévenu

Décision de l'enseignant (sous 48 heures)

Un seul critère bloquant suffit pour que le dossier soit à reprendre ; un dossier à reprendre est corrigé et remis sous 48 heures.

Critère	Type	Respecté	Commentaire
Données accessibles et échantillon présent (L2)	Bloquant	☐	
MVP réaliste en 4 semaines pour cette taille d'équipe (L1, L4)	Bloquant	☐	
Livrables répartis, chacun avec un rédacteur et un relecteur distincts	Bloquant	☐	
Licence et données personnelles traitées (L2)	Bloquant	☐	
Architecture cohérente avec les fonctionnalités (L1, L3)	À améliorer	☐	
Écrans couvrant tout le MVP (L3)	À améliorer	☐	
Tâches courtes, critères de fin vérifiables, charge équilibrée (L4)	À améliorer	☐	
Alternative de repli et jeu de test définis (L2)	À améliorer	☐	

Décision : ☐ Validé ☐ À reprendre ☐ Refusé (changement de sujet)

Date et visa : _____

Lien avec l'évaluation. Ce dossier sert de référence pour le critère P1 (respect du cadrage) et alimente P3 (jeu de test et indicateurs) et P7 (pipeline et suivi). La répartition et les pull requests de chaque membre comptent comme preuves pour le coefficient individuel.